

```

import pandas as pd
import numpy as np
import os

# =====
# SETTINGS
# =====

CSV_FILE = "NQ_data.csv"
OUTPUT_FILE = "mnq_10000_random_results.csv"

CONTRACTS = 4
TICK_SIZE = 0.25
TICKS = 200

POINT_DISTANCE = TICK_SIZE * TICKS
DOLLARS_PER_POINT_PER_CONTRACT = 2.0

TP_SL_DOLLARS = (
    POINT_DISTANCE
    * DOLLARS_PER_POINT_PER_CONTRACT
    * CONTRACTS
)

NUM_SIMULATIONS = 10000
BASE_SEED = 42

# Normal full-session requirement
REQUIRED_START_TIME = "09:30:00"
REQUIRED_END_TIME = "16:00:00"

# =====
# LOAD DATA
# =====

if not os.path.exists(CSV_FILE):
    print(f"ERROR: Could not find {CSV_FILE}")
    exit()

df = pd.read_csv(
    CSV_FILE,
    sep=";",
    header=None,

```

```

names=[
    "date",
    "time",
    "O",
    "H",
    "L",
    "C",
    "volume"
]
)

for col in ["O", "H", "L", "C", "volume"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

df["datetime"] = pd.to_datetime(
    df["date"].astype(str) + " " + df["time"].astype(str),
    errors="coerce"
)

df = df.dropna(
    subset=["datetime", "O", "H", "L", "C"]
)

df = df.sort_values("datetime").reset_index(drop=True)

# =====
# BUILD ELIGIBLE DAYS
# =====
#
# Only trade normal sessions that contain
# both a 9:30 AM bar and a 4:00 PM bar.
#
# Holiday / early-close sessions are excluded.
# =====

eligible_days = []

for date, day in df.groupby(df["datetime"].dt.date):

    day = day.sort_values("datetime").reset_index(drop=True)

    first_time = day.iloc[0]["datetime"].strftime("%H:%M:%S")
    last_time = day.iloc[-1]["datetime"].strftime("%H:%M:%S")

```

```

if (
    first_time == REQUIRED_START_TIME
    and last_time == REQUIRED_END_TIME
):
    eligible_days.append(day)

print("=====")
print("BUILDING ELIGIBLE TRADING DAYS")
print("=====")

print(f"Total dates in dataset: {df['date'].nunique()}")
print(f"Eligible full-session days: {len(eligible_days)}")
print(f"Excluded holiday/early-close days: "
      f"{df['date'].nunique() - len(eligible_days)}")

# =====
# PRECOMPUTE EACH DAY
# =====
#
# For each day we calculate:
#
# LONG:
# +400 if target hit
# -400 if stop hit
# time-exit P&L otherwise
#
# SHORT:
# +400 if target hit
# -400 if stop hit
# time-exit P&L otherwise
#
# This lets us run 10,000 simulations very quickly.
# =====

day_results = []

ambiguous_days = []

for day in eligible_days:

    date = day.iloc[0]["date"]

```

```

entry_bar = day.iloc[0]

entry = entry_bar["O"]
entry_time = entry_bar["datetime"]

long_target = entry + POINT_DISTANCE
long_stop = entry - POINT_DISTANCE

short_target = entry - POINT_DISTANCE
short_stop = entry + POINT_DISTANCE

long_result = None
short_result = None

# Ignore the entry candle itself
post_entry = day[
    day["datetime"] > entry_time
]

# =====
# LONG PATH
# =====

for _, bar in post_entry.iterrows():

    hit_target = bar["H"] >= long_target
    hit_stop = bar["L"] <= long_stop

    if hit_target and hit_stop:

        long_result = "AMBIGUOUS"
        break

    elif hit_target:

        long_result = "WIN"
        break

    elif hit_stop:

        long_result = "LOSS"
        break

```

```

# =====
# SHORT PATH
# =====

for _, bar in post_entry.iterrows():

    hit_target = bar["L"] <= short_target
    hit_stop = bar["H"] >= short_stop

    if hit_target and hit_stop:

        short_result = "AMBIGUOUS"
        break

    elif hit_target:

        short_result = "WIN"
        break

    elif hit_stop:

        short_result = "LOSS"
        break

# =====
# TIME EXIT
# =====

final_close = day.iloc[-1]["C"]

if long_result is None:

    long_pnl = (
        (final_close - entry)
        * DOLLARS_PER_POINT_PER_CONTRACT
        * CONTRACTS
    )

    long_result = "TIME_EXIT"

elif long_result == "WIN":

    long_pnl = TP_SL_DOLLARS

```

```

elif long_result == "LOSS":

    long_pnl = -TP_SL_DOLLARS

else:

    long_pnl = np.nan

if short_result is None:

    short_pnl = (
        (entry - final_close)
        * DOLLARS_PER_POINT_PER_CONTRACT
        * CONTRACTS
    )

    short_result = "TIME_EXIT"

elif short_result == "WIN":

    short_pnl = TP_SL_DOLLARS

elif short_result == "LOSS":

    short_pnl = -TP_SL_DOLLARS

else:

    short_pnl = np.nan

if (
    long_result == "AMBIGUOUS"
    or short_result == "AMBIGUOUS"
):
    ambiguous_days.append(date)

day_results.append({
    "date": date,
    "long_pnl": long_pnl,
    "short_pnl": short_pnl,
    "long_result": long_result,

```

```
    "short_result": short_result
})
```

```
day_results = pd.DataFrame(day_results)
```

```
# =====
# AMBIGUOUS DAY REPORT
# =====
```

```
print(f"Ambiguous days: {len(ambiguous_days)}")
```

```
if len(ambiguous_days) > 0:
```

```
    print("Ambiguous dates:")
    for d in ambiguous_days:
        print(d)
```

```
# =====
# REMOVE AMBIGUOUS DAYS
# =====
#
# We exclude them rather than arbitrarily deciding
# which side of the candle happened first.
# =====
```

```
clean_days = day_results[
    day_results["long_pnl"].notna()
    & day_results["short_pnl"].notna()
].reset_index(drop=True)
```

```
print(f"Final eligible days used: {len(clean_days)}")
```

```
# =====
# RUN 10,000 RANDOM PATHS
# =====
```

```
rng = np.random.default_rng(BASE_SEED)
```

```
long_pnl = clean_days["long_pnl"].to_numpy()
```

```

short_pnl = clean_days["short_pnl"].to_numpy()

simulation_results = []

print("\n=====")
print("RUNNING RANDOM SIMULATIONS")
print("=====")

for simulation in range(1, NUM_SIMULATIONS + 1):

    # 0 = SHORT
    # 1 = LONG
    directions = rng.integers(
        0,
        2,
        size=len(clean_days)
    )

    # Select corresponding P&L
    pnl = np.where(
        directions == 1,
        long_pnl,
        short_pnl
    )

    wins = np.sum(pnl == TP_SL_DOLLARS)
    losses = np.sum(pnl == -TP_SL_DOLLARS)

    time_exits = (
        len(pnl) - wins - losses
    )

    total_pnl = pnl.sum()

    average_pnl = pnl.mean()

    simulation_results.append({

        "simulation": simulation,

        "trades": len(pnl),

        "wins": wins,

```



```

        "losses": losses,

        "time_exits": time_exits,

        "win_rate": (
            wins / (wins + losses) * 100
            if (wins + losses) > 0
            else 0
        ),

        "total_pnl": total_pnl,

        "average_pnl": average_pnl

    })

```

```

results = pd.DataFrame(simulation_results)

```

```

# =====
# SAVE RESULTS
# =====

```

```

results.to_csv(
    OUTPUT_FILE,
    index=False
)

```

```

# =====
# DISTRIBUTION STATISTICS
# =====

```

```

print("\n=====")
print("10,000 RANDOM NULL STRATEGIES")
print("=====")

```

```

print(
    f"Simulations: {len(results):,}"
)

```

```

print(
    f"Trades per simulation: "

```

```

    f"{len(clean_days):,}"
)

print(
    f"Mean total P&L: "
    f"${results.total_pnl.mean():.2f}"
)

print(
    f"Median total P&L: "
    f"${results.total_pnl.median():.2f}"
)

print(
    f"Std. dev. total P&L: "
    f"${results.total_pnl.std():.2f}"
)

print(
    f"5th percentile: "
    f"${results.total_pnl.quantile(.05):.2f}"
)

print(
    f"95th percentile: "
    f"${results.total_pnl.quantile(.95):.2f}"
)

print(
    f"Mean average trade: "
    f"${results.average_pnl.mean():.4f}"
)

print(
    f"Median average trade: "
    f"${results.average_pnl.median():.4f}"
)

print(
    f"Mean TP/SL win rate: "
    f"${results.win_rate.mean():.2f}%"
)

print(

```

```
    f"Profitable simulations: "  
    f"{(results.total_pnl > 0).sum():,} "  
    f"{((results.total_pnl > 0).mean() * 100:.2f}%"  
)  
  
print(  
    f"Losing simulations: "  
    f"{(results.total_pnl < 0).sum():,} "  
    f"{((results.total_pnl < 0).mean() * 100:.2f}%"  
)  
  
print(  
    f"Exactly zero: "  
    f"{(results.total_pnl == 0).sum():,}"  
)  
  
print("\nResults saved to:")  
print(OUTPUT_FILE)
```